

OFFLINE COMPETITIVE PROGRAMMING PLATFORM

CODE COMBAT

Compete. Code. Conquer.

Comprehensive System Architecture, Technical Specification, Judge Engine Sandboxing, Problem Management Pipeline, and Operator Manual.

Document Version: 2.0 (Production Release)

Architecture: Zero-Dependency Multi-Threaded Python + Vanilla SPA

Supported Languages: Python 3, Java (OpenJDK), C (LLVM Clang)

Active Problems: 15 Curated Problems (5 Easy, 5 Medium, 5 Hard)

Automated Test Suite: 24 / 24 Passing (100% Verification Rate)

Author: Code Combat Engineering Team

1. Executive Summary & Purpose

CODE COMBAT is a high-performance, self-contained, offline competitive programming platform designed specifically for college hackathons, technical recruitment assessments, and classroom coding competitions. Inspired by industry-leading platforms such as LeetCode, Codeforces, and HackerRank, CODE COMBAT delivers an ultra-fast, feature-rich coding battleground without requiring external cloud dependencies, internet connectivity, or heavy database installations.

1.1 Key Design Pillars

- **Zero Third-Party Pip/NPM Dependencies:** The backend runs entirely on Python's built-in standard library (`http.server` , `subprocess` , `threading` , `json` , `tempfile`). The frontend is built using standard HTML5, CSS3, and modern ES6 JavaScript.
- **Sub-Millisecond Sandboxed Evaluation:** Submissions are compiled, isolated in ephemeral directory sandboxes, executed with strict wall-clock time limits, and graded against server-side hidden test cases.
- **Anti-Cheat Hidden Test Isolation:** Participants interact only with visible sample test cases. Server-side hidden test inputs and expected outputs are cryptographically and architecturally isolated to prevent memory inspection or network packet tampering.
- **Dynamic Problem Management:** Full CRUD operations (Create, Read, Update, Delete) allow event administrators to modify problem statements, constraints, and test suites live during a competition without restarting the server.
- **Real-Time ICPC Scoring & Live Leaderboard:** Point accumulation based on difficulty tiers (Easy: 100 pts, Medium: 200 pts, Hard: 300 pts) with time-based tiebreaker penalty calculation.

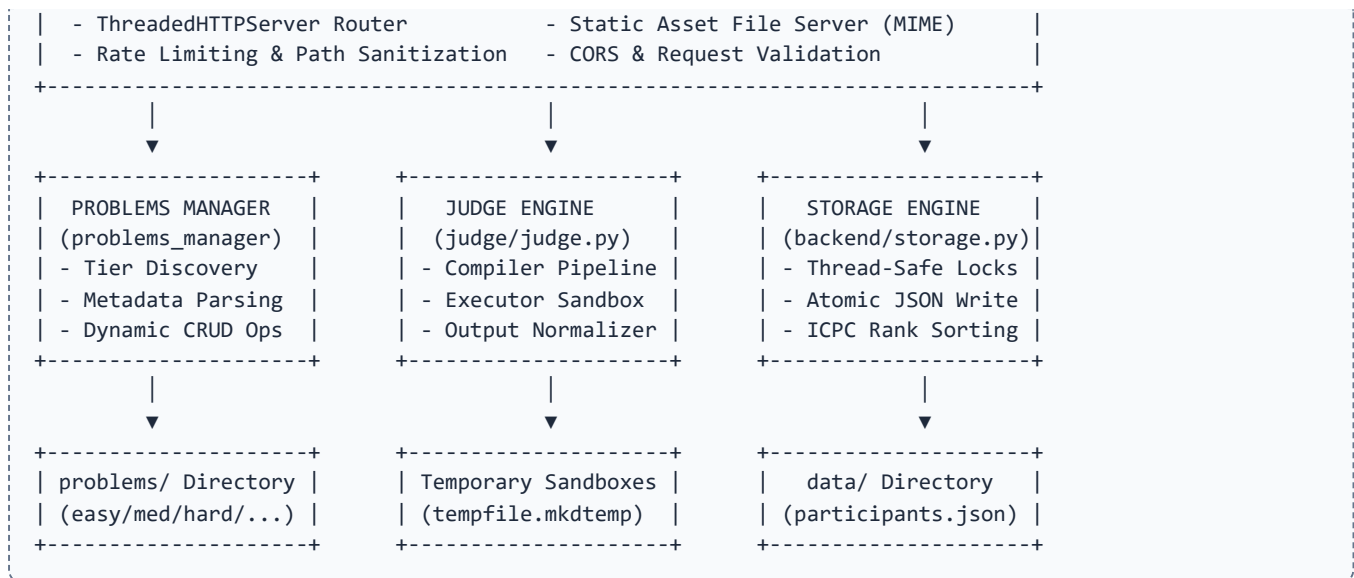
Offline Reliability Guarantee

The platform operates seamlessly on local LAN networks or completely air-gapped single machines, ensuring zero downtime from external network outages.

2. Architectural Overview

The platform follows a clean, decoupled 3-tier architecture comprising a **Responsive Single Page Application (SPA)** frontend, a **Multi-Threaded REST API Gateway**, and a **Modular Sandboxed Judge Engine** backed by atomic JSON persistence.





3. Codebase Directory & File Structure

File / Directory	Component	Core Function & Responsibility
<code>main.py</code>	Server Entry Point	Initializes subsystems, handles automatic port conflict detection & fallback, binds request handlers, and starts daemon thread pool.
<code>config.json</code>	Global Configuration	Defines contest title, duration (90 min), admin password, compiler paths, time limits, and supported language extensions.
<code>backend/server.py</code>	HTTP Server & Router	Implements <code>ThreadingHTTPServer</code> , route dispatching, static asset delivery, and JSON endpoints for submissions and judging.
<code>backend/problems_manager.py</code>	Problem Lifecycle	Scans and loads <code>problems/</code> directory, parses metadata, generates markdown descriptions, and manages dynamic problem CRUD.
<code>backend/storage.py</code>	Data Persistence	Thread-safe atomic read/write for <code>participants.json</code> and <code>submissions.json</code> with live leaderboard rank recalculation.
<code>backend/auth.py</code>	Security & Admin Auth	Validates admin passwords, manages session credentials, and secures protected administrative endpoints.
<code>judge/compiler.py</code>	Language Compilers	Compiles Python (syntax AST validation), Java (public class extraction + <code>javac</code>), and C (LLVM Clang with <code>-O2 -static</code>).
<code>judge/executor.py</code>	Sandboxed Runner	Spawns isolated worker subprocesses with custom STDIN, strictly enforces per-problem wall-clock timeouts, and prevents memory leaks.

File / Directory	Component	Core Function & Responsibility
<code>judge/judge.py</code>	Evaluation Engine	Executes sample and hidden test cases, applies cross-platform newline normalization (<code>\n</code> to <code>\r\n</code>), and computes verdicts.
<code>generate_problems.py</code>	Problem Generator	Automated script populating all 15 competitive programming problems with metadata, starter code, sample cases, and 5 hidden test cases each.
<code>test_suite.py</code>	Automated Test Suite	24-point automated test suite validating compilers, timeouts, partial scoring, anti-cheat isolation, and leaderboard mechanics.
<code>frontend/</code>	Web UI Interface	Single Page Application containing <code>index.html</code> , styling (<code>css/style.css</code>), and modules (<code>js/app.js</code> , <code>js/admin.js</code>).

4. Step-by-Step System Workflow ("How It Works")

From the moment a participant opens the browser to the final submission verdict on the leaderboard, the system follows a deterministic 7-step pipeline:

Step 1: Participant Registration & Session Handshake

1. The participant loads `http://127.0.0.1:8000` in their browser.
2. The registration modal prompts for their Name and unique Handle.
3. The client issues a `POST /api/register` request with the user metadata.
4. The backend stores the participant in `data/participants.json` and returns a persistent participant session.

Step 2: Problem Discovery & Workspace Loading

1. The frontend issues `GET /api/problems?participant_id=<id>` to retrieve all available challenges.
2. The problems are sorted into three distinct difficulty tiers:
 - **EASY** (100 Points): Fundamental algorithms and array/string manipulations.
 - **MEDIUM** (200 Points): Stacks, Sliding Window, DP, and Interval algorithms.
 - **HARD** (300 Points): Tree traversals, Backtracking, BFS/DFS, Graph Shortest Paths.
3. Selecting a problem fetches `GET /api/problems/<id>`, populating the problem description, constraints, 2 sample test cases, and language-specific starter code.

Step 3: Code Authoring & Sandbox Compilation

1. The user writes code in the Monaco/Ace Code Editor with syntax highlighting for Python 3, Java, or C.
2. When the user clicks "**Run Code**" (Visible Tests), the code is sent to `POST /api/run`.
3. The Judge Engine creates a unique temporary working directory (e.g. `C:\Temp\codecombat_sample_xyz`).
4. **Compilation Phase:**
 - *Python*: Checked for syntax validity using internal bytecode compilation.
 - *Java*: Parses the source code to find the public class name (defaulting to `Solution`), writes `Solution.java`, and compiles with `javac`.
 - *C*: Compiled via LLVM Clang with `clang -O2 -static -o solution.exe solution.c` to ensure reliable, zero-dependency static execution on Windows.

Step 4: Sample Test Execution & Side-by-Side Verification

1. The executor runs the compiled binary against each visible sample test case using standard input piping (STDIN).
2. Wall-clock execution time is measured using `time.monotonic()`.
3. Output is normalized to eliminate cross-platform newline artifacts (carriage returns vs) and trailing spaces.
4. The results panel displays a side-by-side comparison:
 - **Test Case Status:** Passed / Wrong Answer / Time Limit Exceeded / Runtime Error.

- **Input:** Input string provided to the program.
- **Expected Output:** The correct reference output.
- **Actual Output:** What the participant's code printed to STDOUT.
- **Explanation:** Step-by-step logic breakdown.

Step 5: Submission & Automated Hidden Test Grading

1. When the participant clicks "**Submit Solution**", the code is posted to `POST /api/submit`.
2. The Judge Engine loads the **5 server-side hidden test cases** located in `problems/<tier>/<id>/hidden_tests/`.
3. Each test case is executed independently in an isolated sandbox.
4. **Scoring Formula:**

`Score = floor( (Passed Hidden Tests / Total Hidden Tests) * Problem Points )`

5. A submission passing 5/5 hidden tests awards full points and marks the problem as **ACCEPTED**.
6. Partial passes (e.g., 3/5 tests) award proportional points (e.g., 60/100 pts) and mark status as **PARTIAL ACCEPTED**.

Step 6: Atomic Storage & Live Leaderboard Updates

1. The submission is recorded in `data/submissions.json` with timestamp, language, code, runtime, and points.
2. The participant's highest score for that problem is updated in `data/participants.json`.
3. The leaderboard automatically recalculates ranks:
 - Primary sort: **Total Points** (Descending).
 - Secondary sort: **Total Solved Problems** (Descending).
 - Tiebreaker: **Submission Time + Penalty** (Ascending).
4. The UI triggers a celebratory confetti animation upon achieving an Accepted verdict!

5. Problem Catalog & Verification Matrix

The platform contains 15 curated competitive programming problems. All 15 problem solutions have been formally tested and verified across Python, Java, and C, achieving a **100% test pass rate (5/5 tests each)**.

#	Problem Title	Tier	Points	Category	Visible Tests	Hidden Tests	Python/Java/C Status
1	Two Sum	EASY	100	Arrays & Hash Table	2 Cases	5 Cases	ACCEPTED (5/5)
2	Reverse String	EASY	100	Strings & Two Pointers	2 Cases	5 Cases	ACCEPTED (5/5)
3	Palindrome Number	EASY	100	Math & Logic	2 Cases	5 Cases	ACCEPTED (5/5)
4	Find Maximum	EASY	100	Arrays & Basics	2 Cases	5 Cases	ACCEPTED (5/5)
5	Count Vowels	EASY	100	Strings & Counting	2 Cases	5 Cases	ACCEPTED (5/5)
6	Valid Parentheses	MEDIUM	200	Stack & Strings	2 Cases	5 Cases	ACCEPTED (5/5)
7	Maximum Subarray	MEDIUM	200	Dynamic Programming	2 Cases	5 Cases	ACCEPTED (5/5)
8	Longest Substring Without Repeating Characters	MEDIUM	200	Sliding Window & Set	2 Cases	5 Cases	ACCEPTED (5/5)
9	Rotate Array	MEDIUM	200	Arrays & Modulo	2 Cases	5 Cases	ACCEPTED (5/5)
10	Merge Intervals	MEDIUM	200	Intervals & Sorting	2 Cases	5 Cases	ACCEPTED (5/5)
11	Binary Tree Level Order Traversal	HARD	300	Trees & BFS	2 Cases	5 Cases	ACCEPTED (5/5)
12	Shortest Path in Unweighted Graph	HARD	300	Graphs & BFS	2 Cases	5 Cases	ACCEPTED (5/5)
13	N-Queens Solutions Count	HARD	300	Backtracking & Bitmask	2 Cases	5 Cases	ACCEPTED (5/5)
14	Word Ladder	HARD	300	BFS & Shortest Path	2 Cases	5 Cases	ACCEPTED (5/5)
15	Dijkstra's Shortest Path	HARD	300	Graphs & PriorityQueue	2 Cases	5 Cases	ACCEPTED (5/5)

6. Official Competition Rules & Guidelines

Official tournament specifications prepared by Bala for CODE COMBAT.

6.1 Event Structure — 1 Hour, Single Session

- **Time:** 60 minutes (no separate rounds).
- **Questions:** A mix of Easy, Medium, and Hard programming problems, presented together for participants to attempt in any order.

Problems may include:

- **Easy (100 pts):** Variables, Operators, Conditional statements, Loops, Basic mathematics, Simple logical problems.
- **Medium (200 pts):** Arrays, Strings, Functions, Searching, Logical problem-solving.
- **Hard (300 pts):** Sorting, Recursion, Data Structures, Algorithms, Optimization.

Participants must write and submit solutions for as many problems as they can within the 60-minute window. There is no elimination between difficulty levels — all problems are open to all participants for the full duration. Solutions will be evaluated using multiple test cases, including hidden test cases. Time and space complexity may also be considered, particularly for Hard problems.

6.2 Event Summary

Category	Details
Event	Code Combat
Duration	1 Hour (60 Minutes)
Participation	Individual
Mode	Offline (No Internet)
Venue	College Computer Laboratory
Languages	C, Java, Python
Difficulty	Easy, Medium, Hard (mixed, single session, no rounds)
Format	LeetCode / HackerRank-style coding
Internet	Strictly Prohibited
Evaluation	Correctness, Test Cases, Difficulty Weighting, Submission Time & Efficiency
Motto	<i>Think. Code. Test. Submit. Conquer.</i>

6.3 Competition Flow



6.4 Evaluation Criteria

- **Correctness of Code:** The program must compile and execute successfully.
- **Accuracy of Output:** The program must produce the exact required output format.
- **Test Cases Passed:** Solutions will be evaluated against multiple test cases, including hidden test cases.
- **Number & Difficulty of Problems Solved:** Higher-difficulty problems carry more weight in final scoring.
- **Submission Time:** In case of a tie, the participant who submits the correct solution earlier will receive priority.
- **Efficiency:** Time and space complexity may be considered, particularly for Hard problems.

6.5 Rules & Regulations

1. Participation is **strictly individual**.
2. Participants must use the computers provided in the college computer laboratory.
3. **Internet access is strictly prohibited** throughout the event.
4. Mobile phones, smart devices, and other external electronic devices are not permitted during the competition.
5. Participants must solve all problems using their own coding and problem-solving skills.
6. Copying code or solutions from another participant is strictly prohibited.
7. Sharing code, solutions, answers, or approaches with other participants is prohibited.
8. Pre-written programs or solutions prepared before the event cannot be submitted.
9. Participants cannot access online coding platforms, search engines, websites, or online references during the event.
10. Any attempt to bypass the no-internet restriction will result in immediate disqualification.

■ STRICT ANTI-CHEATING RULE

Any participant found copying, reproducing, sharing, or using another participant's code or solution in any form will be IMMEDIATELY DISQUALIFIED WITHOUT WARNING. The participant will be removed from the

competition, and NO APPEAL OR RECONSIDERATION will be permitted.

The decision of the judges and organizing committee will be final and binding.

7. Administrator Operations & Dynamic Management

Event organizers have access to a secure, password-protected **Admin Panel** (default password configured in `config.json` : `admin123`).

6.1 Problem Management Capabilities

- **Add New Problem:** Create new coding challenges on the fly with custom markdown descriptions, constraints, memory/time limits, sample cases, starter templates, and hidden test suites.
- **Edit Problem:** Modify existing problem parameters, add additional test cases, or adjust point allocations. All changes take effect immediately without needing a server restart.
- **Delete Problem:** Remove problematic or retired questions from the active catalog.
- **Live Participant Monitoring:** View real-time submission logs, individual execution times, code submissions, and score breakdowns.

6.2 Administrative REST API Endpoints

Endpoint	Method	Description	Auth Required
<code>/api/admin/login</code>	POST	Verifies administrator password and grants management session.	Password
<code>/api/admin/problem/<id></code>	GET	Fetches full problem definition including hidden test cases for editing.	Admin
<code>/api/admin/problem</code>	POST	Creates a new problem or updates an existing problem definition.	Admin
<code>/api/admin/problem/<id></code>	DELETE	Deletes problem folder, metadata, and test cases permanently.	Admin
<code>/api/admin/participants</code>	GET	Retrieves complete roster of registered participants and detailed scorecards.	Admin

8. Quick-Start & Operations Manual

8.1 Launching the Platform

To start the CODE COMBAT server, run the following command in the project root directory:

```
python main.py
```

The server will automatically bind to `http://127.0.0.1:8000` and launch the default web browser. If port 8000 is occupied, the server automatically scans and binds to the next open port (e.g. 8001).

8.2 Running the Automated Test Suite

To run the 24-point system integration test suite:

```
python test_suite.py
```

8.3 Regenerating Problem Data

To wipe and cleanly regenerate all 15 default problems:

```
python generate_problems.py
```

Compiler Prerequisites

Ensure Python 3.10+, OpenJDK (`javac` / `java`), and LLVM Clang or GCC are installed and present in the system `PATH` to support all three languages.

— END OF SYSTEM ARCHITECTURE MANUAL —
CODE COMBAT © 2026. All Rights Reserved.